CSE4203: Computer Graphics

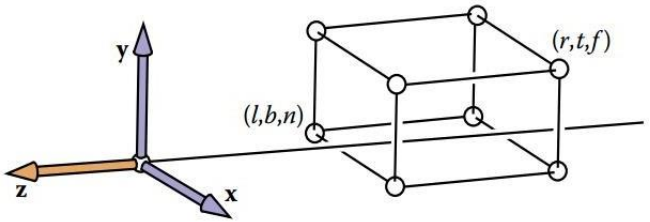
Lecture – 5 (part - B)

Viewing

Outline

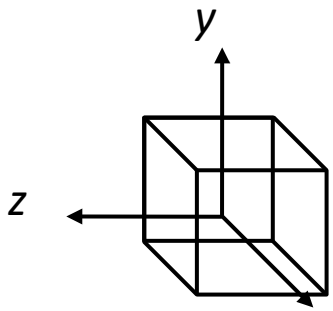
- Coordinate Transformation
- Camera Transformation

Recap (1/1)



Orthographic view volume

M_{orth}



Canonical view volume

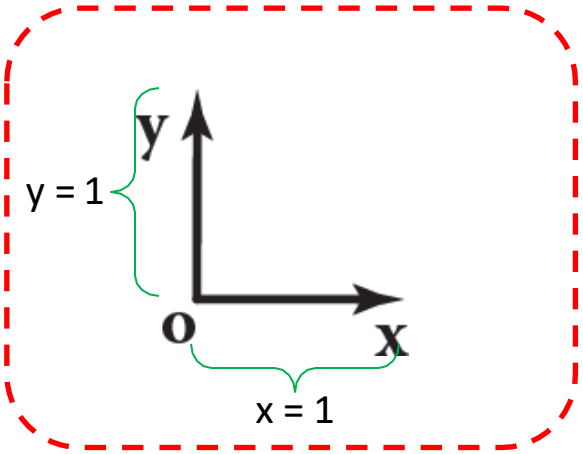
M_{vp}



Screen

Coordinate System (1/1)

- A coordinate system, or coordinate frame, consists of **an origin** and **a basis**: *a set of three (two for 2D) orthonormal vectors.*



2D example

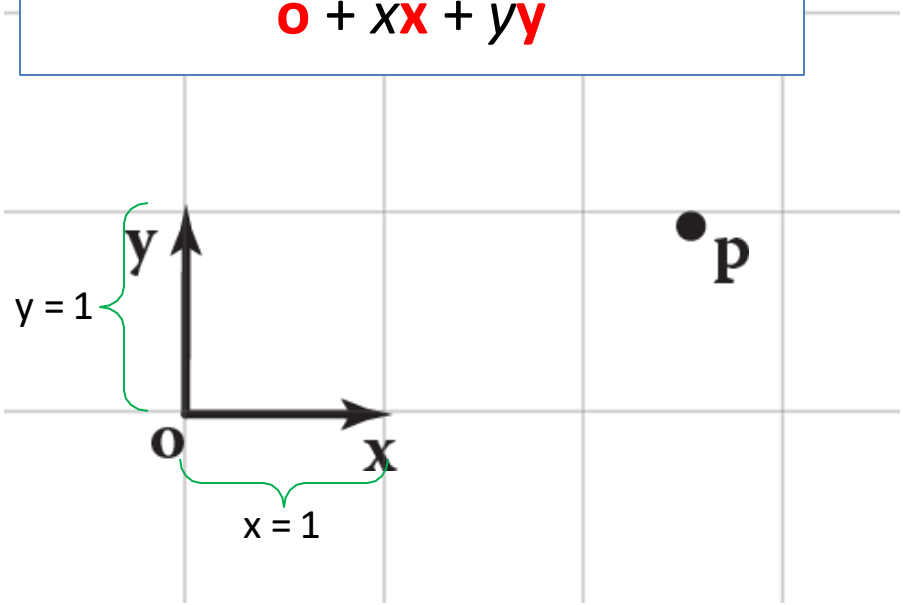
Canonical coordinate system:

- origin **o**
- orthonormal basis vectors **{x, y}**.
- Also called: *World* coordinates

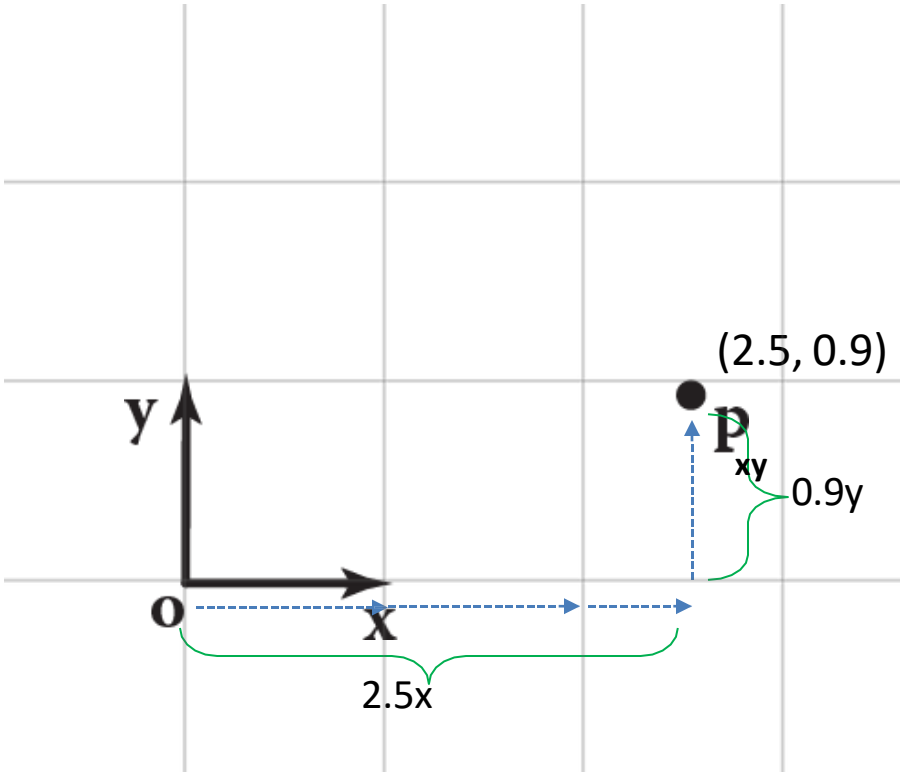
Coordinate Transformation (1/20)

In a frame with origin **o** and basis {**x, y**}, the coordinates (*x, y*) describe the point:

o + *x***x** + *y***y**



Coordinate Transformation (4/20)

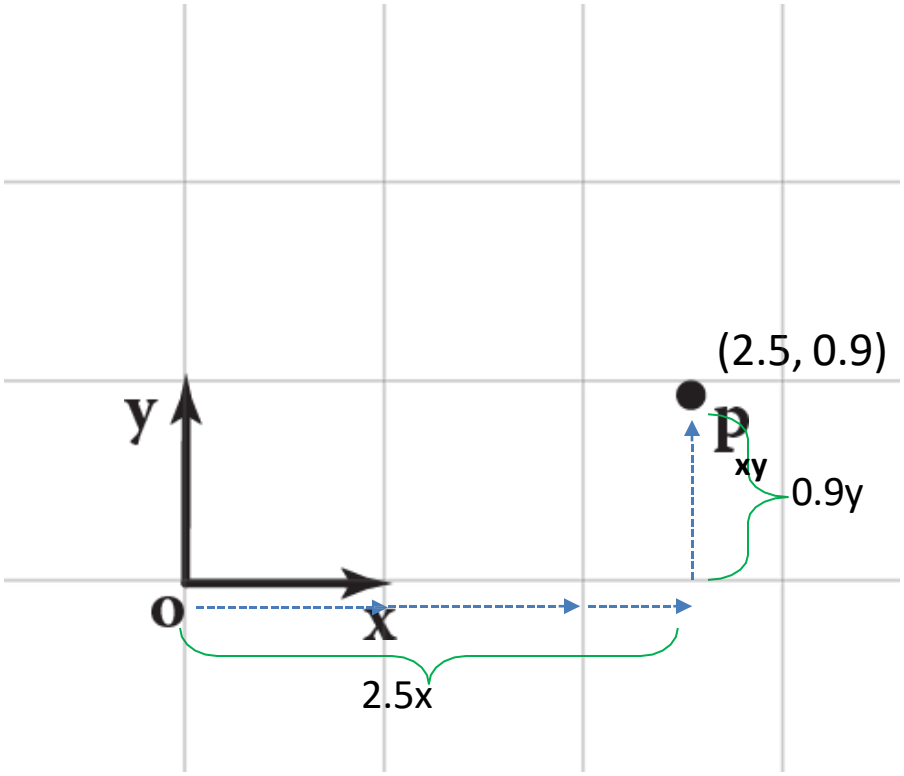


$$\mathbf{p_{xy}} = (x_p, y_p) \equiv \mathbf{o} + x_p\,\mathbf{x} + y_p\,\mathbf{y}$$

$$\begin{pmatrix} x_p \\ y_p \end{pmatrix} = \begin{pmatrix} ? \\ ? \end{pmatrix} + x_p \begin{pmatrix} ? \\ ? \end{pmatrix} + y_p \begin{pmatrix} ? \\ ? \end{pmatrix}$$

$$= \begin{pmatrix} 2.5 \\ 0.9 \end{pmatrix}$$

Coordinate Transformation (5/20)



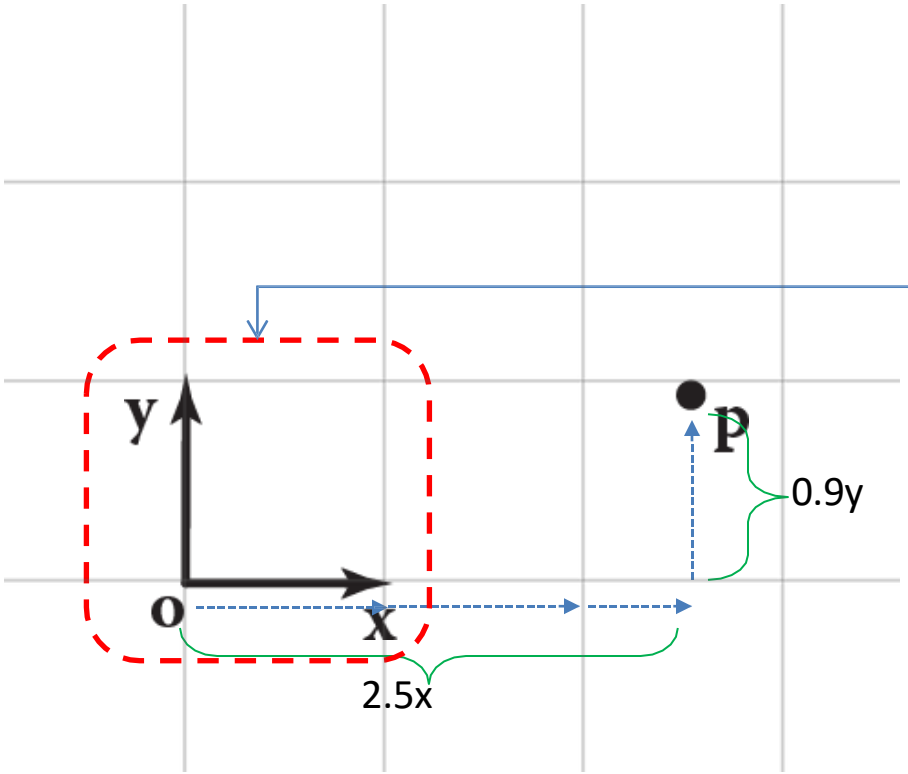
$$\mathbf{p_{xy}} = (x_p, y_p) \equiv \mathbf{o} + x_p\,\mathbf{x} + y_p\,\mathbf{y}$$

$$\begin{pmatrix} x_p \\ y_p \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + 2.5 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + 0.9 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

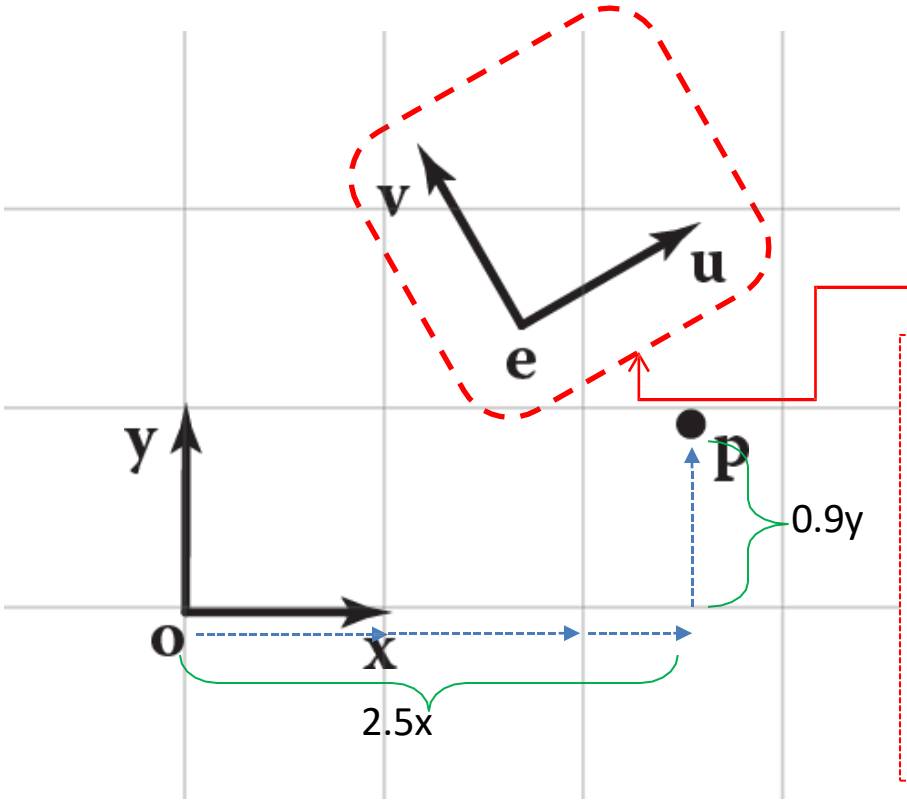
$$= \begin{pmatrix} 2.5 \\ 0.9 \end{pmatrix}$$

Coordinate Transformation (6/20)

Canonical coordinate system is the **frame-of-reference** for all other coordinate systems.



Coordinate Transformation (7/20)



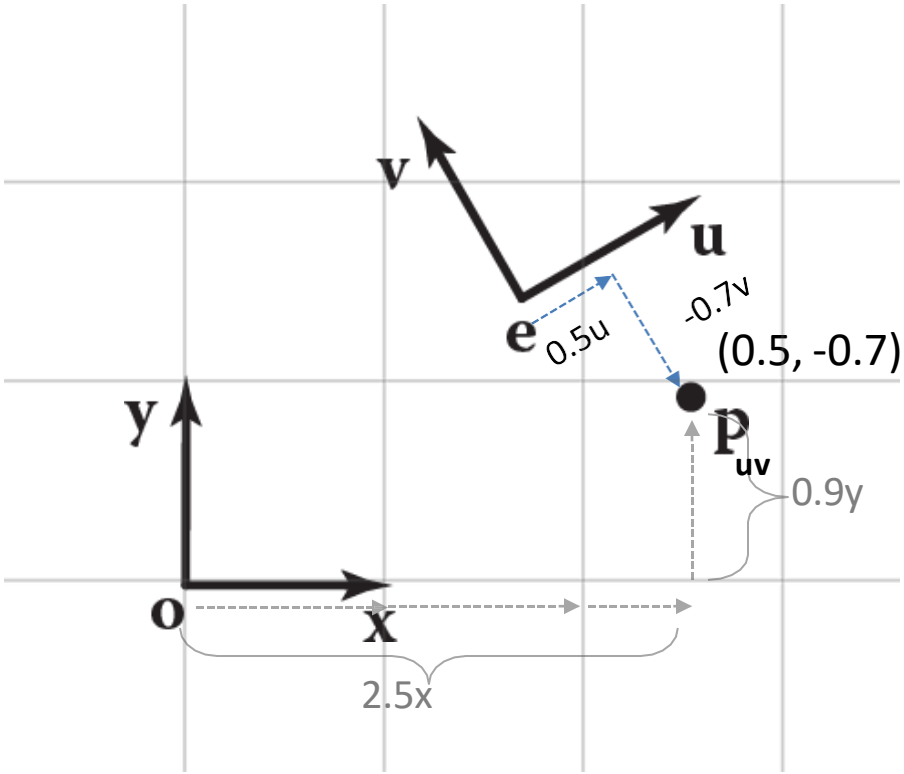
Canonical coordinate system is the frame-of-reference for all other coordinate systems.

Another coordinate system (**frame coordinate**):

- **within canonical coord.**
- origin **e**
- orthonormal basis vectors **{u, v}**

$\mathbf{p_{uv}} = (u_p, v_p) \equiv \mathbf{e} + u_p\,\mathbf{u} + v_p\mathbf{v}$

Coordinate Transformation (8/20)

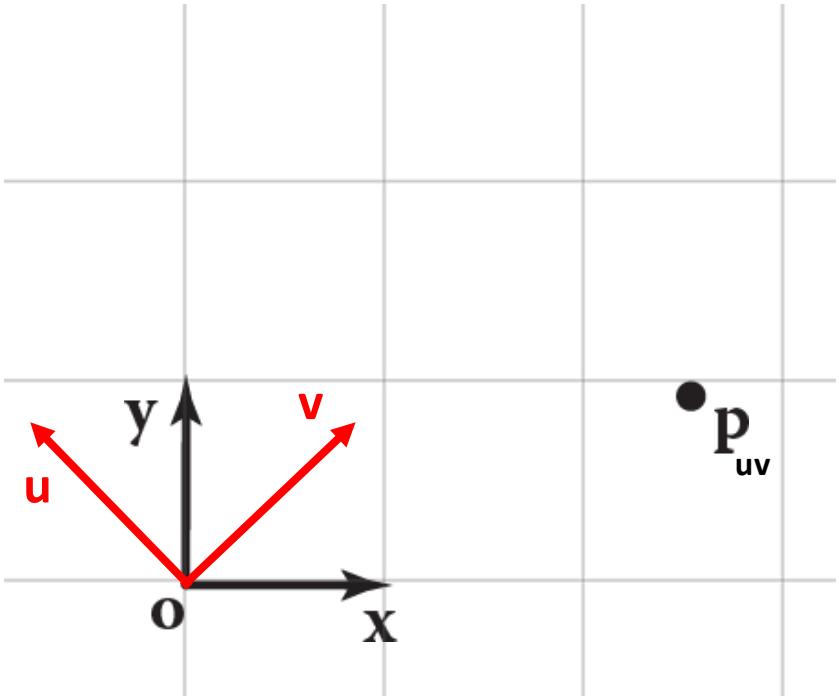


$$\mathbf{p_{uv}} = (u_p, v_p) \equiv \mathbf{e} + u_p\,\mathbf{u} + v_p\mathbf{v}$$

$$\begin{pmatrix} u_p \\ v_p \end{pmatrix} = \begin{pmatrix} x_e \\ y_e \end{pmatrix} + 0.5\begin{pmatrix} x_u \\ y_u \end{pmatrix} + (-0.7)\begin{pmatrix} x_v \\ y_v \end{pmatrix}$$

$$= \begin{pmatrix} 0.5 \\ -0.7 \end{pmatrix}$$

Coordinate Transformation (8/20)



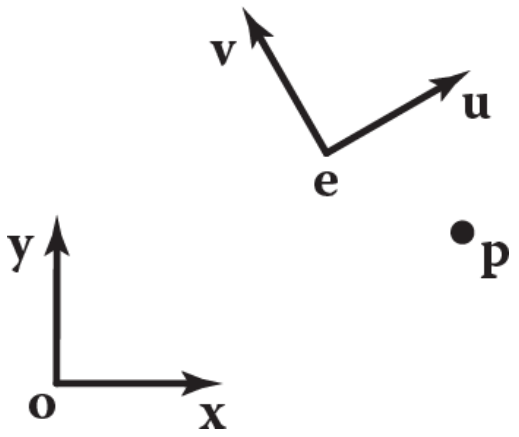
$$\mathbf{p_{uv}} = (u_p, v_p) \equiv \mathbf{e} + u_p \mathbf{u} + v_p \mathbf{v}$$

Q: Suppose the basis vectors of a frame coordinate {**e, u, v**} is achieved by rotating the basis vectors of the canonical coordinate system {**o, x, y**} by 45°. ***Determine the basis vectors of the frame coordinate system.***

Coordinate Transformation (9/20)

$\mathbf{p_{xy}} = (x_p, y_p) \equiv \mathbf{o} + x_p\,\mathbf{x} + y_p\,\mathbf{y}$

$\mathbf{p_{uv}} = (u_p, v_p) \equiv \mathbf{e} + u_p\,\mathbf{u} + v_p\,\mathbf{v}$

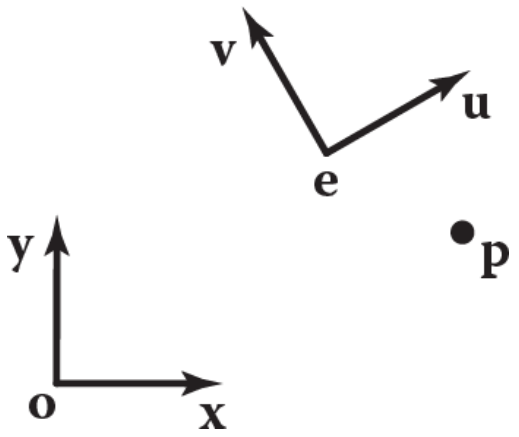


Coordinate Transformation (10/20)

$\mathbf{p_{xy}} = (x_p, y_p) \equiv \mathbf{o} + x_p\,\mathbf{x} + y_p\,\mathbf{y}$

$\mathbf{p_{uv}} = (u_p, v_p) \equiv \mathbf{e} + u_p\,\mathbf{u} + v_p\,\mathbf{v}$

$\mathbf{p_{xy}} \leftrightarrow \mathbf{p_{uv}}$

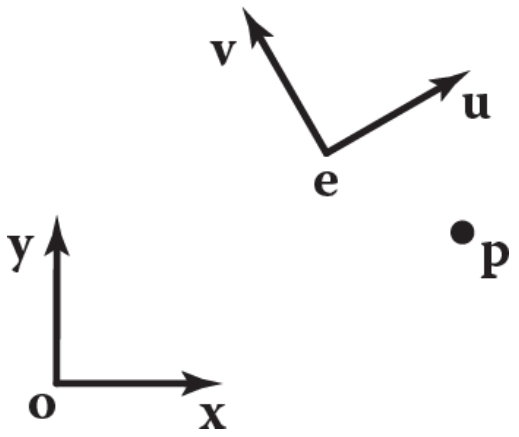


Coordinate Transformation (11/20)

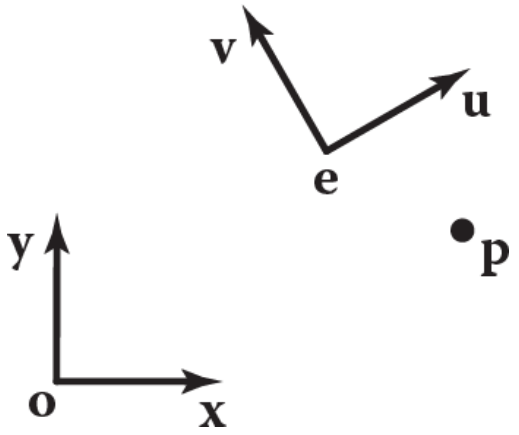
$\mathbf{p_{xy}} = (x_p, y_p) \equiv \mathbf{o} + x_p\,\mathbf{x} + y_p\,\mathbf{y}$

$\mathbf{p_{uv}} = (u_p, v_p) \equiv \mathbf{e} + u_p\,\mathbf{u} + v_p\,\mathbf{v}$

$\mathbf{p_{xy}} = \mathbf{e} + u_p\,\mathbf{u} + v_p\,\mathbf{v}$



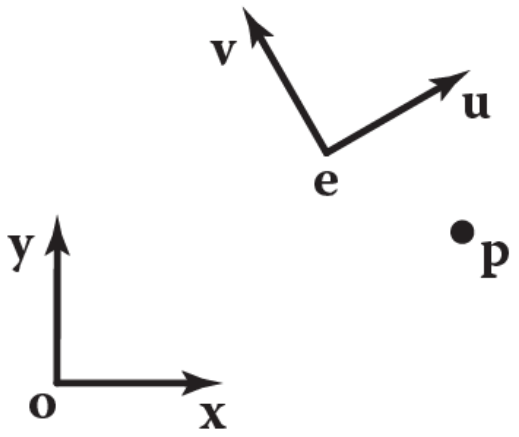
Coordinate Transformation (12/20)



$\mathbf{p}_{xy} = \mathbf{e} + u_p \mathbf{u} + v_p \mathbf{v}$

$$\begin{bmatrix} x_p \\ y_p \end{bmatrix} = \begin{bmatrix} x_e \\ y_e \end{bmatrix} + u_p \begin{bmatrix} x_u \\ y_u \end{bmatrix} + v_p \begin{bmatrix} x_v \\ y_v \end{bmatrix}$$

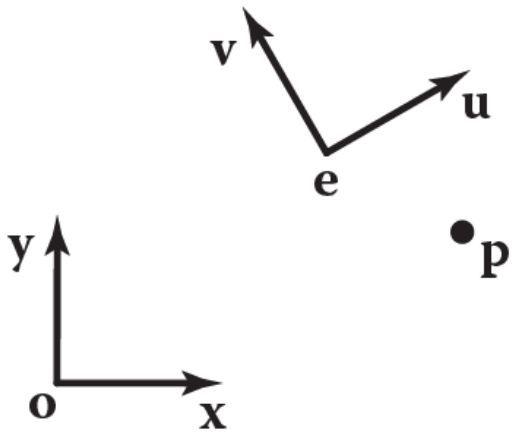
Coordinate Transformation (13/20)



$$\mathbf{p}_{xy} = \mathbf{e} + u_p \mathbf{u} + v_p \mathbf{v}$$

$$\begin{bmatrix} x_p \\ y_p \end{bmatrix} = \begin{bmatrix} x_e \\ y_e \end{bmatrix} + \begin{bmatrix} x_u & x_v \\ y_u & y_v \end{bmatrix} \begin{bmatrix} u_p \\ v_p \end{bmatrix}$$

Coordinate Transformation (14/20)



$$\mathbf{p}_{xy} = \mathbf{e} + u_p\,\mathbf{u} + v_p\,\mathbf{v}$$

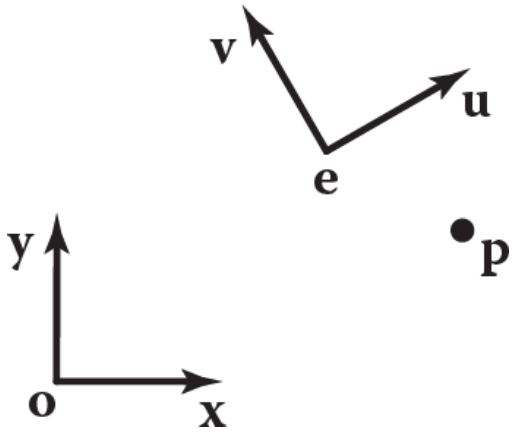
$$\begin{bmatrix} x_p \\ y_p \end{bmatrix} = \begin{bmatrix} x_e \\ y_e \end{bmatrix} + \begin{bmatrix} x_u & x_v \\ y_u & y_v \end{bmatrix} \begin{bmatrix} u_p \\ v_p \end{bmatrix}$$

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_e \\ 0 & 1 & y_e \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_u & x_v & 0 \\ y_u & y_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_p \\ v_p \\ 1 \end{bmatrix}$$

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Coordinate Transformation (15/20)

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} x_u & x_v & x_e \\ y_u & y_v & y_e \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_p \\ v_p \\ 1 \end{bmatrix}$$

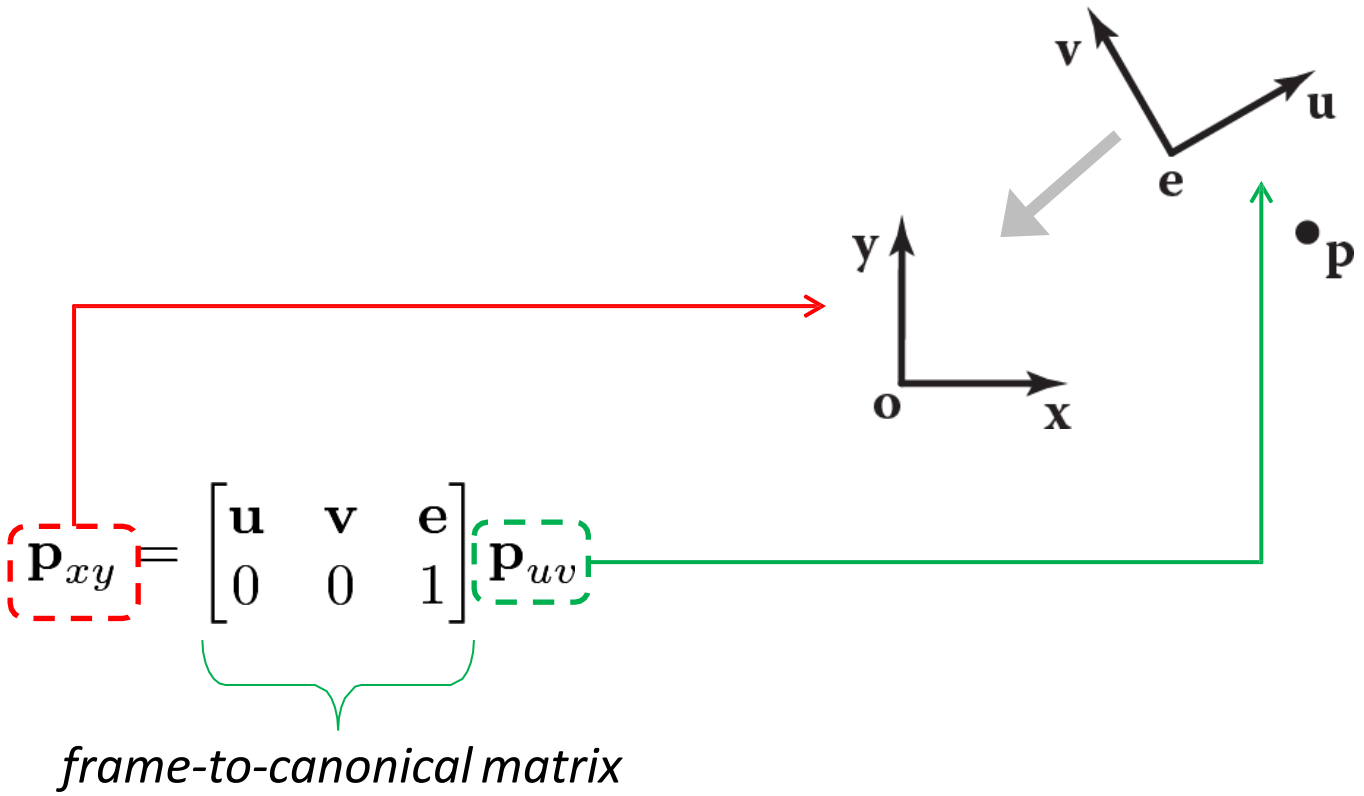


$$\mathbf{p}_{xy} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{e} \\ 0 & 0 & 1 \end{bmatrix} \mathbf{p}_{uv}$$

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_e \\ 0 & 1 & y_e \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_u & x_v & 0 \\ y_u & y_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_p \\ v_p \\ 1 \end{bmatrix}$$

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

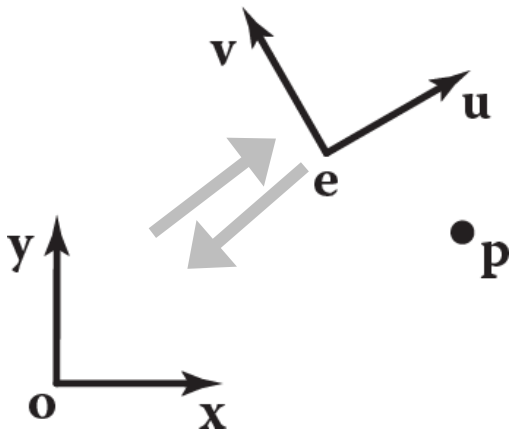
Coordinate Transformation (18/20)



Coordinate Transformation (19/20)

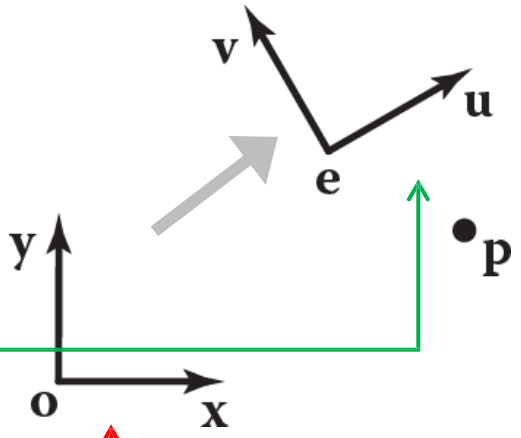
$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_e \\ 0 & 1 & y_e \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_u & x_v & 0 \\ y_u & y_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_p \\ v_p \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} u_p \\ v_p \\ 1 \end{bmatrix} = \begin{bmatrix} x_u & y_u & 0 \\ x_v & y_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_e \\ 0 & 1 & -y_e \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix}$$



Coordinate Transformation (20/20)

$$\begin{bmatrix} u_p \\ v_p \\ 1 \end{bmatrix} = \begin{bmatrix} x_u & y_u & 0 \\ x_v & y_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_e \\ 0 & 1 & -y_e \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix}$$



$$\mathbf{p}_{uv} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{e} \\ 0 & 0 & 1 \end{bmatrix}^{-1} \mathbf{p}_{xy}$$

canonical-to-frame matrix

3D Coordinate Transformation (2/2)

$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & x_e \\ 0 & 1 & 0 & y_e \\ 0 & 0 & 1 & z_e \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_u & x_v & x_w & 0 \\ y_u & y_v & y_w & 0 \\ z_u & z_v & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_p \\ v_p \\ w_p \\ 1 \end{bmatrix}$$

$$\mathbf{p}_{xyz} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{p}_{uvw},$$

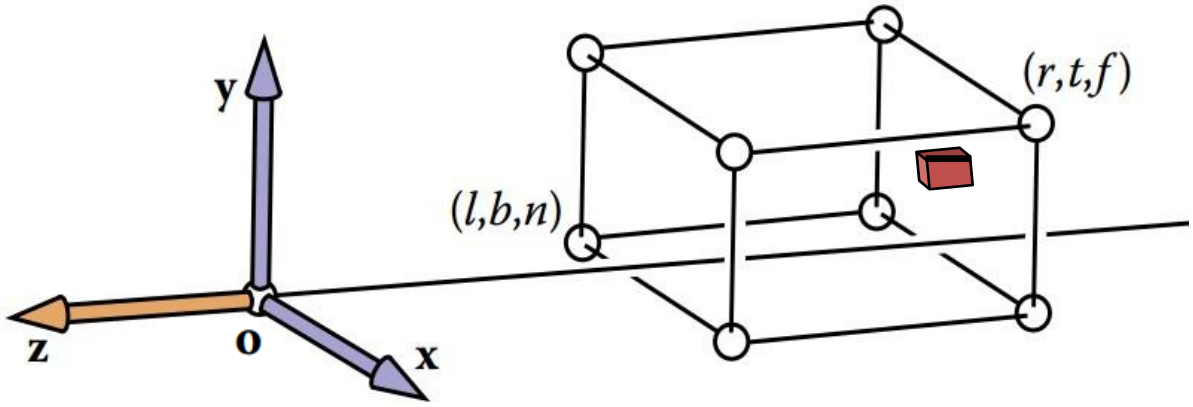
$$\begin{bmatrix} u_p \\ v_p \\ w_p \\ 1 \end{bmatrix} = \begin{bmatrix} x_u & y_u & z_u & 0 \\ x_v & y_v & z_v & 0 \\ x_w & y_w & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_e \\ 0 & 1 & 0 & -y_e \\ 0 & 0 & 1 & -z_e \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix}$$

$$\mathbf{p}_{uvw} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} \mathbf{p}_{xyz}.$$

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

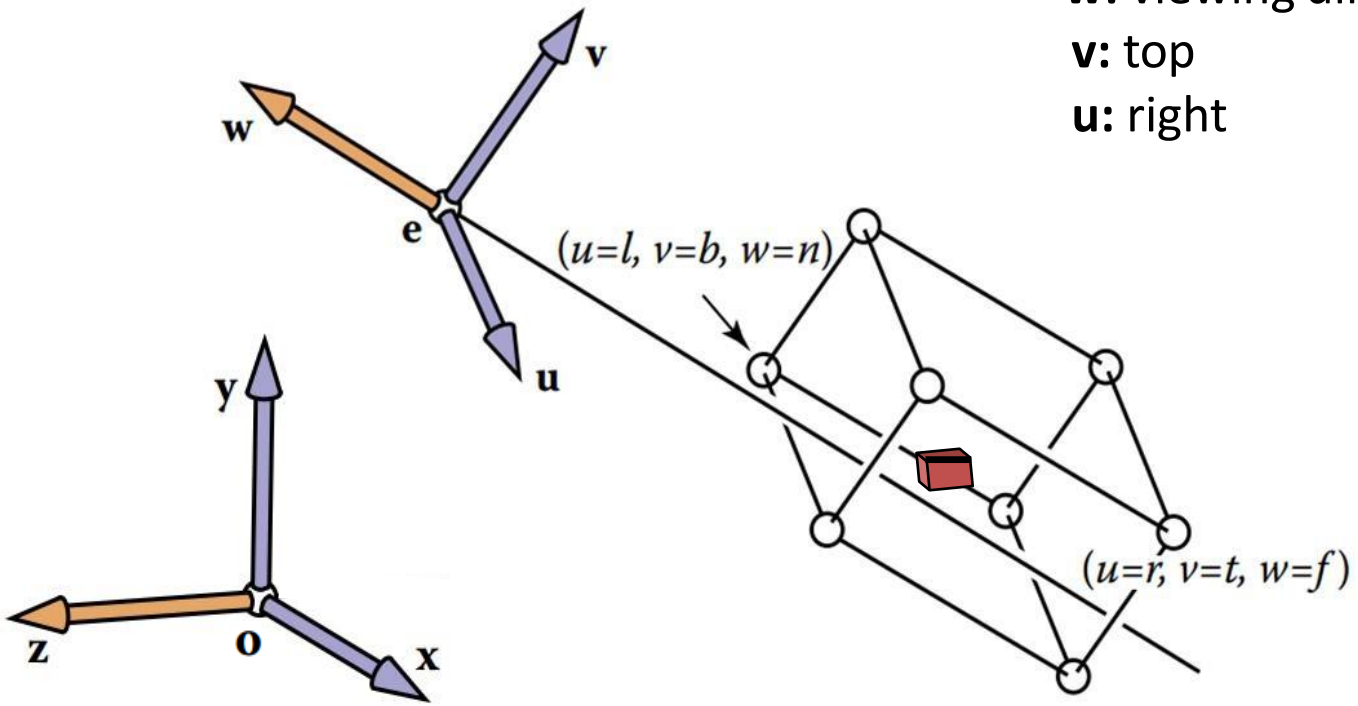
Camera Transformation (2/6)

- We'd like to be able to change the viewpoint in 3D and look in any direction.



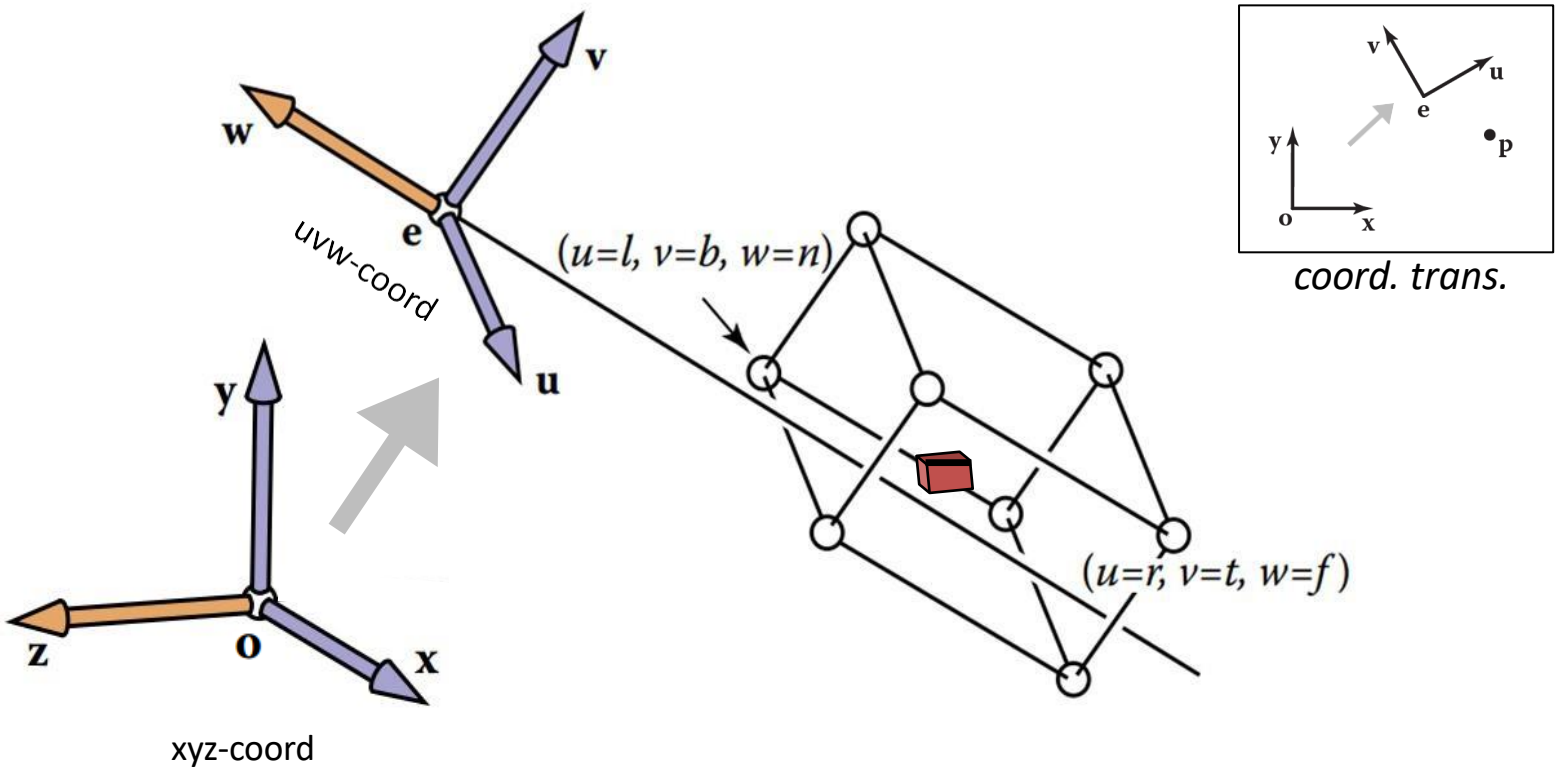
Camera Transformation (3/6)

e: camera/eye position
- **w:** viewing direction
v: top
u: right



Camera Transformation (4/6)

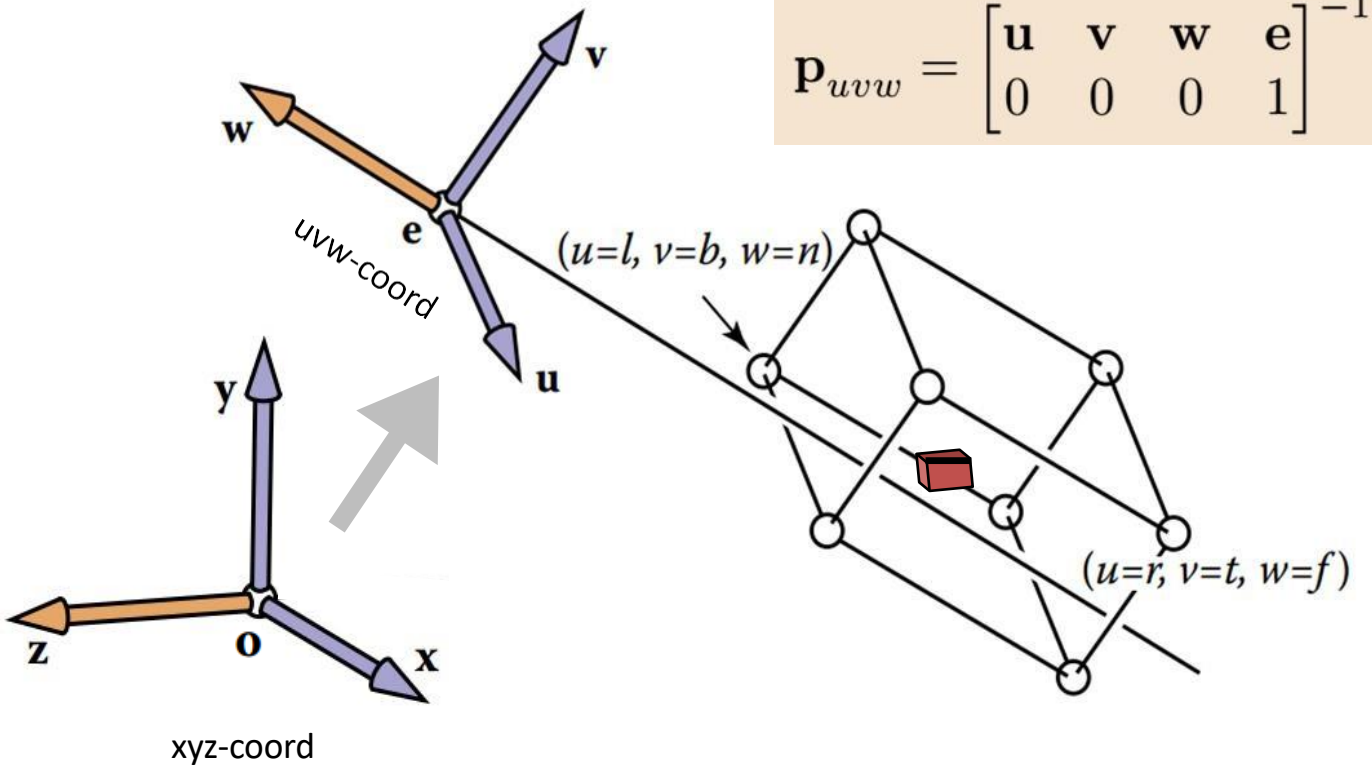
from **xyz-coordinates** into **uvw-coordinates**



Camera Transformation (5/6)

from **xyz-coordinates** into **uvw-coordinates**

$$\mathbf{p}_{uvw} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} \mathbf{p}_{xyz}.$$



Camera Transformation (6/6)

$$\mathbf{p}_{uvw} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} \mathbf{p}_{xyz}.$$

canonical-to-frame matrix:

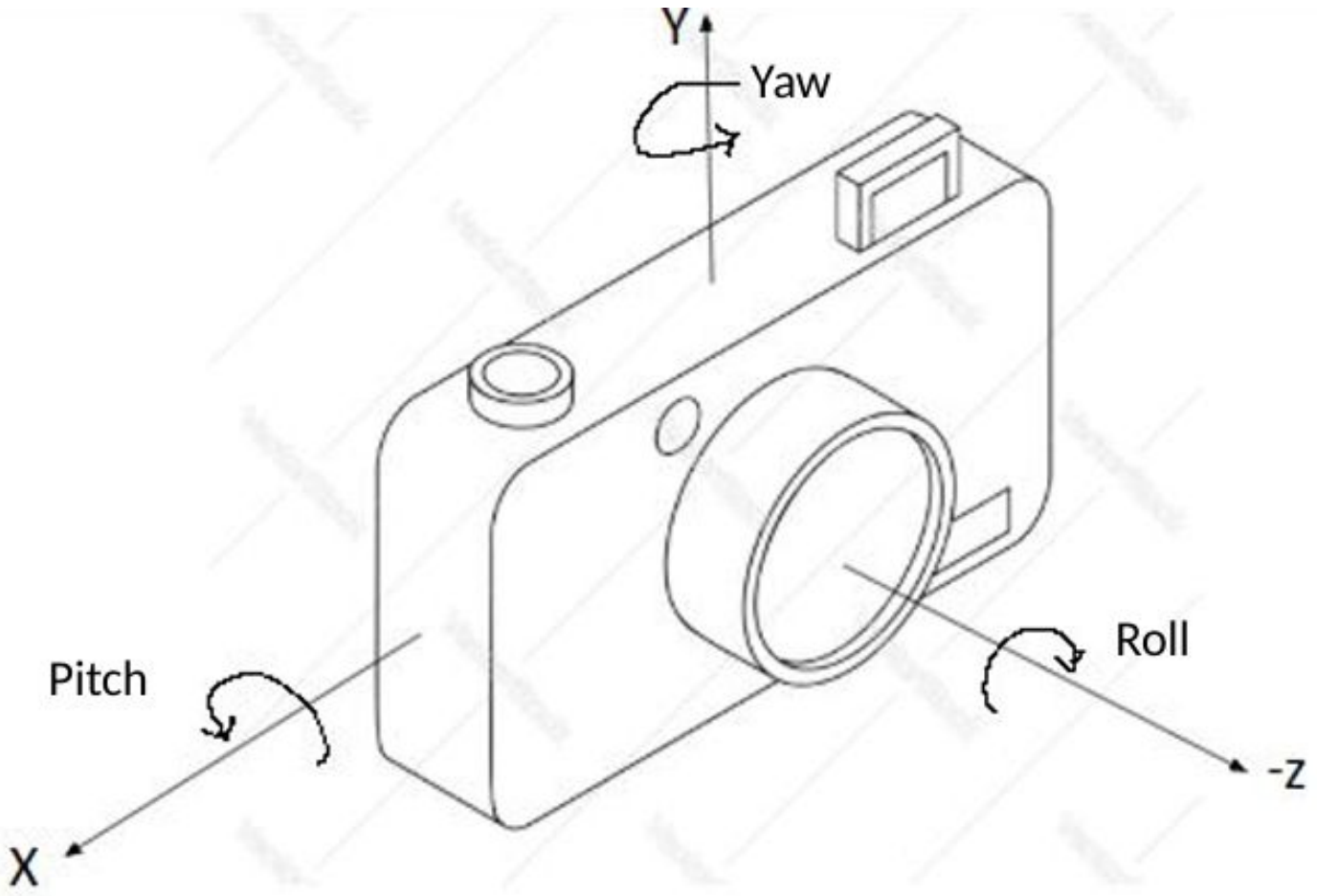
$$\mathbf{M}_{\text{cam}} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} x_u & y_u & z_u & 0 \\ x_v & y_v & z_v & 0 \\ x_w & y_w & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_e \\ 0 & 1 & 0 & -y_e \\ 0 & 0 & 1 & -z_e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Roll–Pitch–Yaw

- Like any other rigid body, a camera in 3D space has three rotational degrees of freedom (X, Y, Z).
- A common convention to rotate a camera is Roll–Pitch–Yaw (RPY) representation.

Roll–Pitch–Yaw



Visualization: https://ursinusgraphics.github.io/RollPitchYaw/

Roll–Pitch–Yaw

- Roll (γ): Rotation around the viewing axis (-z). Tilting the camera sideways.
- Pitch (β): Rotation around the horizontal axis (x). Looking "up" or "down".
- Yaw (α): Rotation around the vertical axis (y). Changes the "heading".

Roll–Pitch–Yaw

To find the final orientation, we multiply the individual rotation matrices to find the new basis.

Order matters (Yaw → Pitch → Roll):

Roll–Pitch–Yaw

Yaw → Pitch → Roll

$$M_{basis} = R_z(\gamma)R_x(\beta)R_y(\alpha)$$

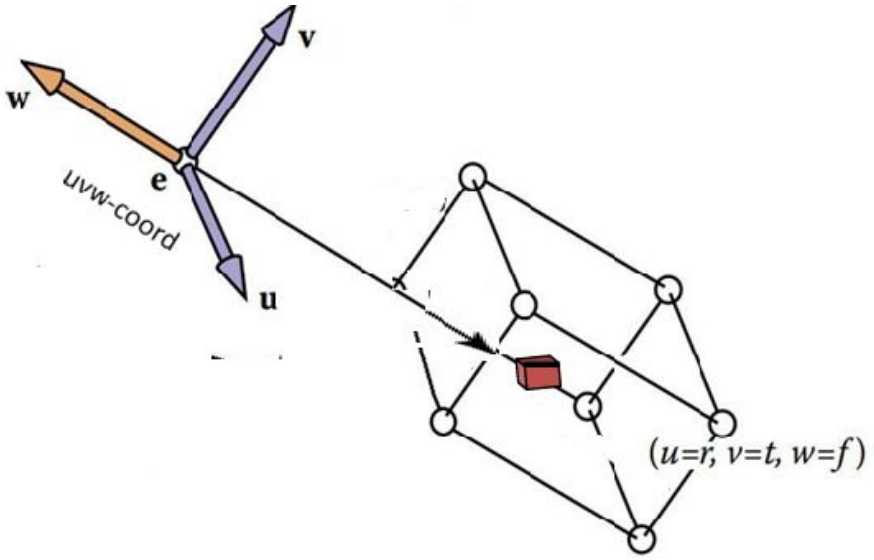
$$= \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 & 0 \\ \sin\gamma & \cos\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\beta & -\sin\beta & 0 \\ 0 & \sin\beta & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\alpha & 0 & \sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\alpha & 0 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} x_u & y_u & z_u & 0 \\ x_v & y_v & z_v & 0 \\ x_w & y_w & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{M}_{\text{cam}} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} x_u & y_u & z_u & 0 \\ x_v & y_v & z_v & 0 \\ x_w & y_w & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_e \\ 0 & 1 & 0 & -y_e \\ 0 & 0 & 1 & -z_e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

LookAt Transformation

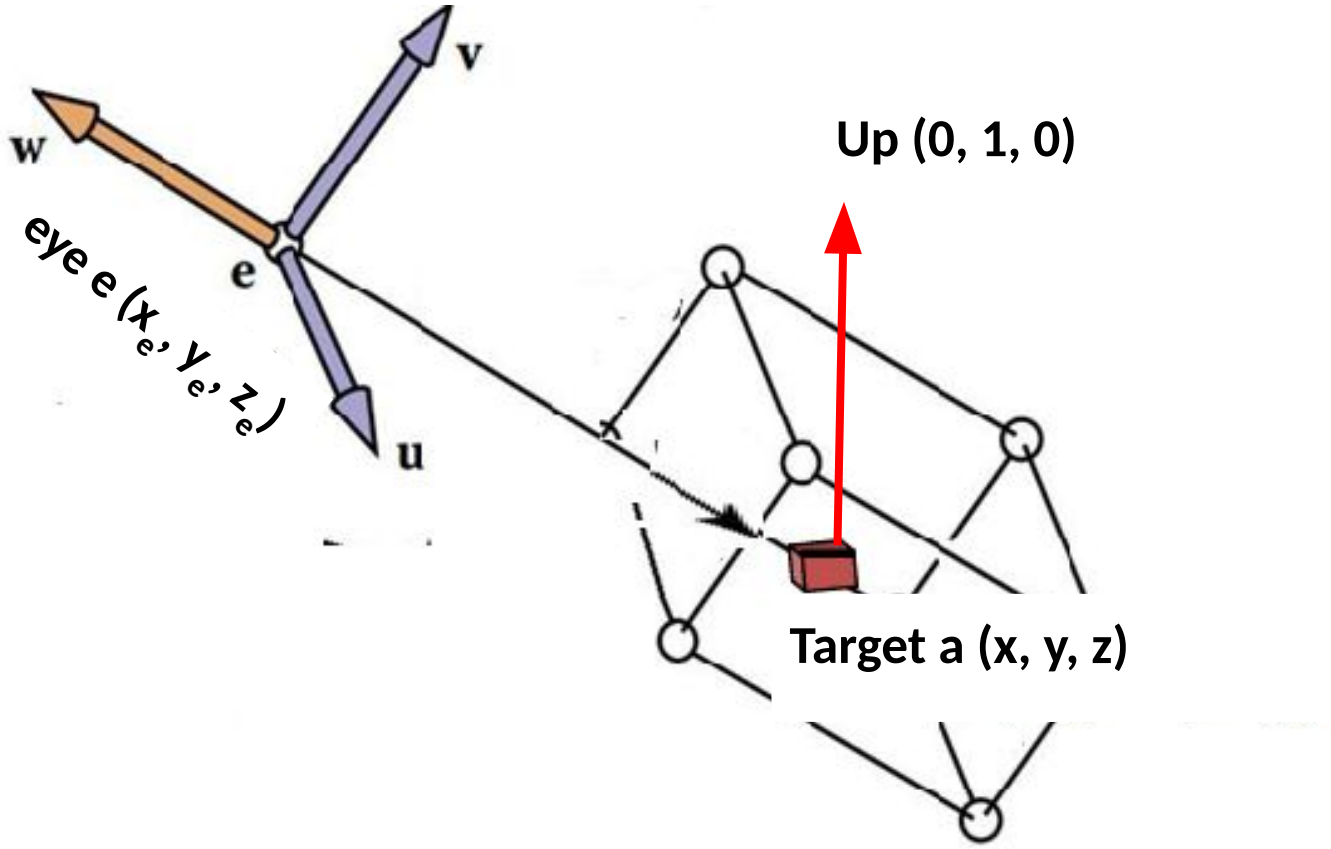
- In most applications (games, CAD), it is easier to say "I am at point e and looking at object a" than using the angles (Roll-Pitch-Yaw)



LookAt Transformation

- To define the frame, we need three pieces of information:
  - **Eye (e):** Camera position in world space.
  - **Target (a):** The point we want to look at.
  - **World-Up (up)**: A reference vector (usually (0, 1, 0) to help define "which way is up."

LookAt Transformation

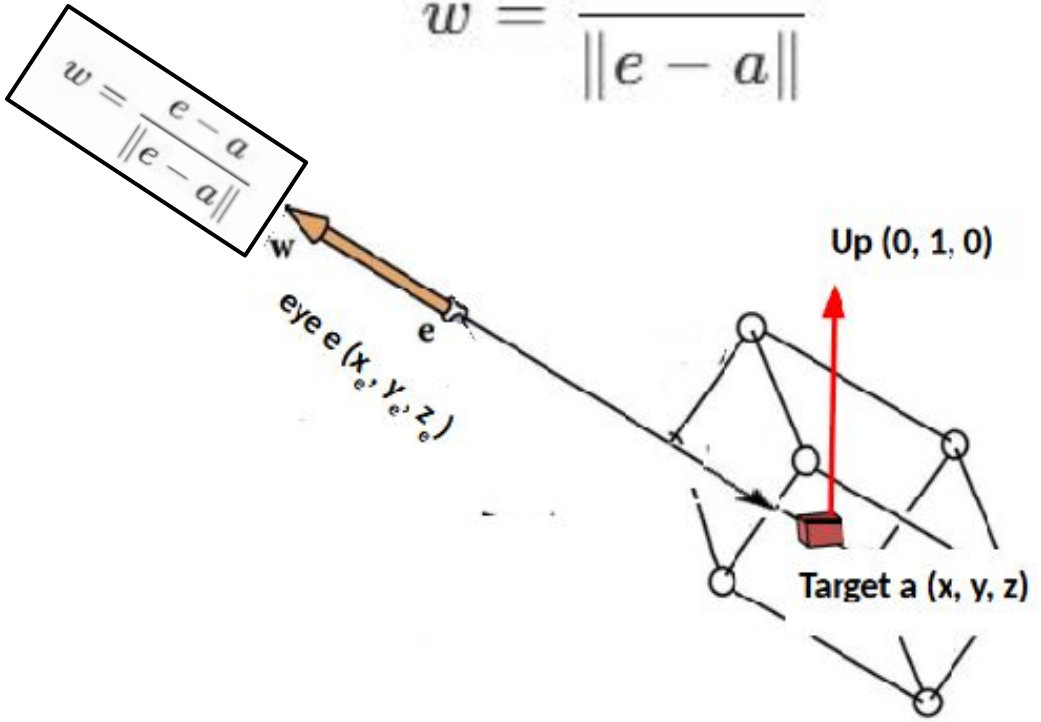


LookAt Transformation

Constructing the Orthonormal Basis vector {u, v, w}:

1. **w (Opposite Viewing Direction):**

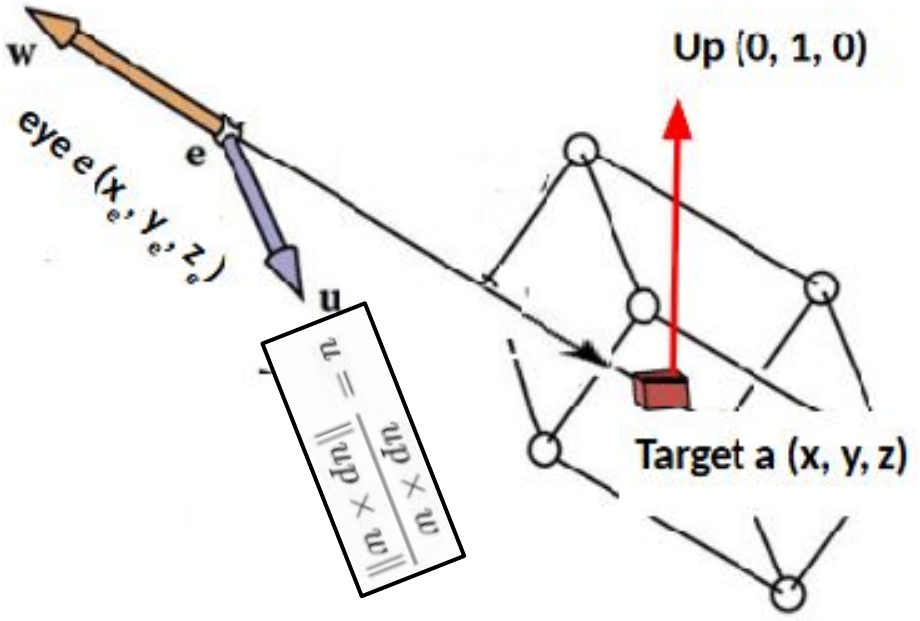
$$w = \frac{e - a}{\|e - a\|}$$



LookAt Transformation

2. u (The Right Axis): It must be perpendicular to both the "up" direction and the view direction.

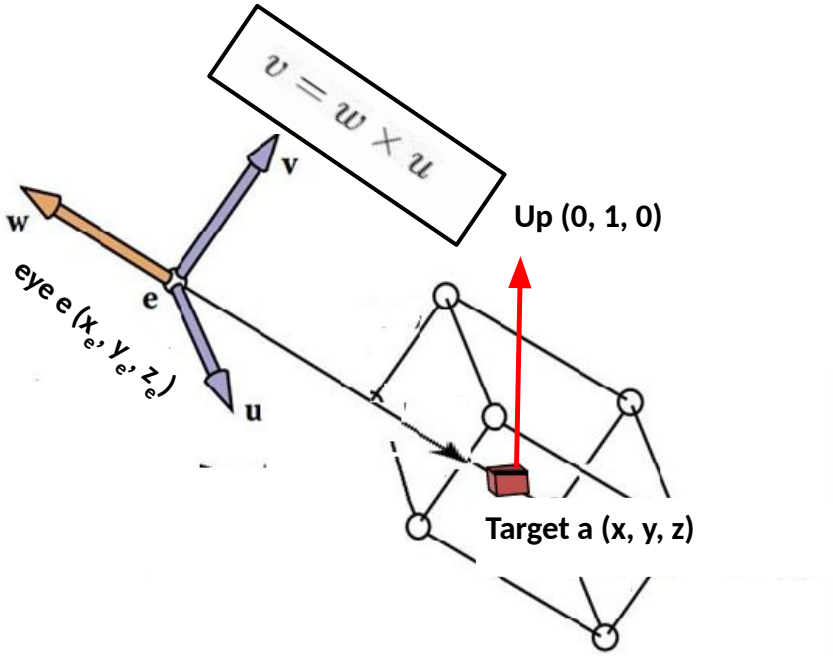
$$u = \frac{up \times w}{\|up \times w\|}$$



LookAt Transformation

3. v (True Up): We find the **true up** vector by crossing the view and right vectors.

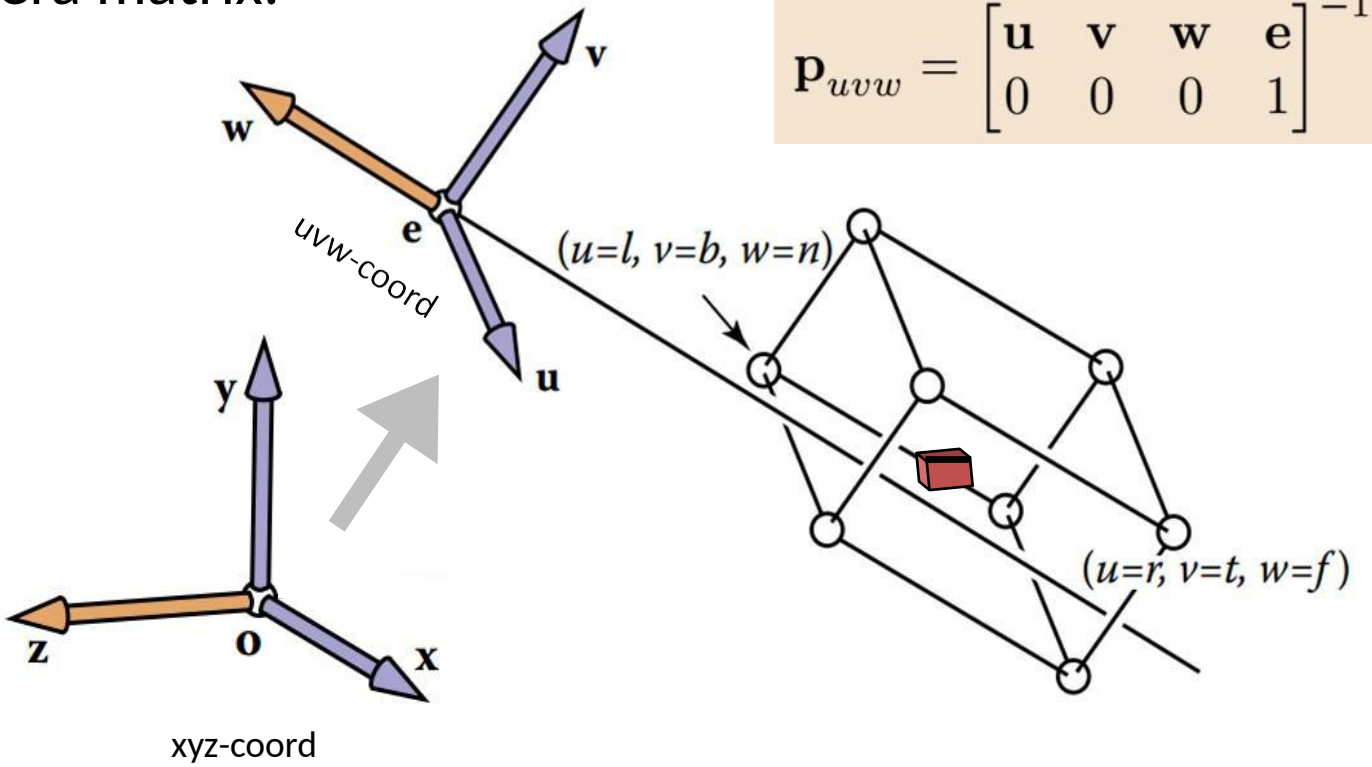
$$v = w \times u$$



LookAt Transformation

Once Basis (u, v, w) are found, they are plugged directly into the camera matrix:

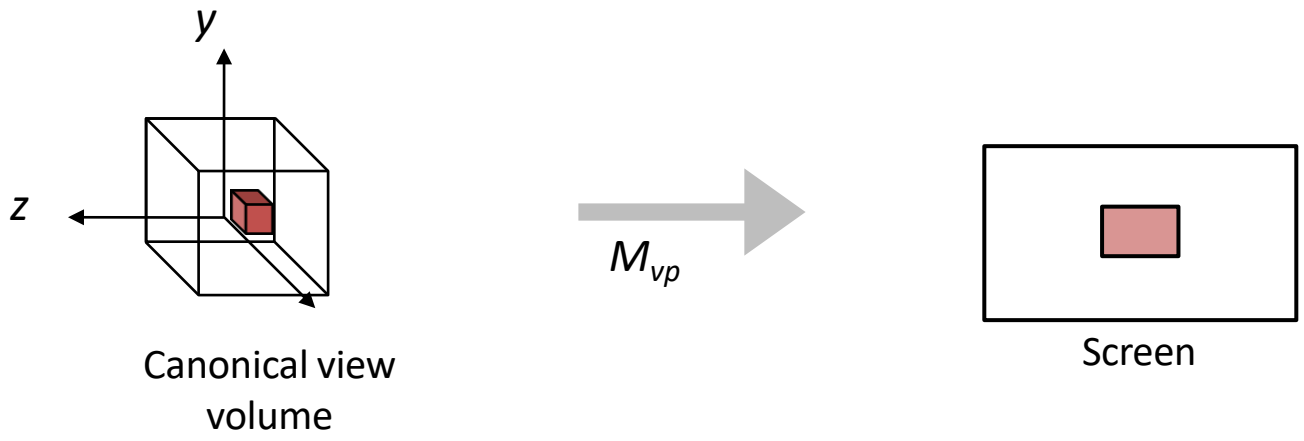
$$\mathbf{p}_{uvw} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} \mathbf{p}_{xyz}.$$



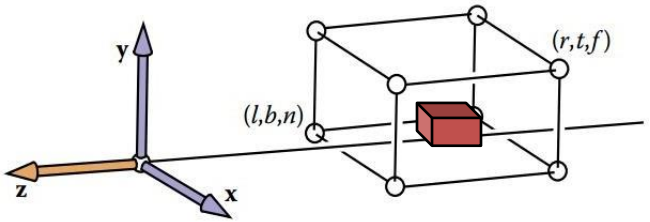
Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

https://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/lookat-function/framing-lookat-function.html

Summary (1/5)

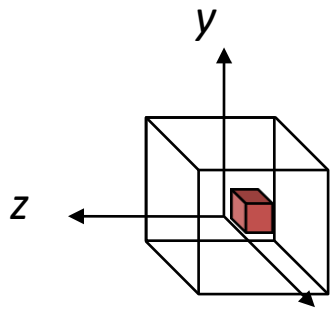


Summary (2/5)



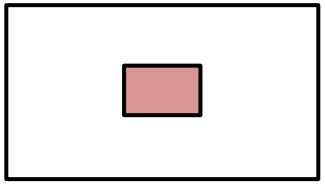
Orthographic view volume

M_{orth}



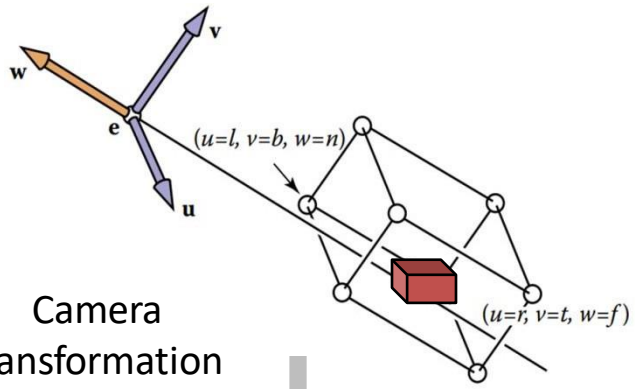
Canonical view volume

M_{vp}



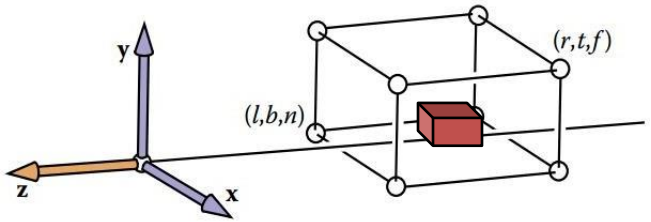
Screen

Summary (3/5)



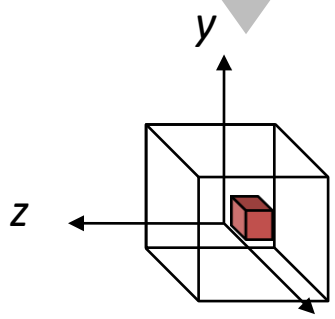
Camera transformation

M_{cam}



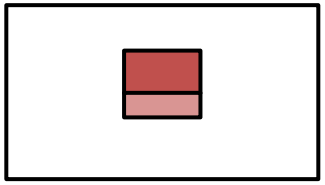
Orthographic view volume

M_{orth}



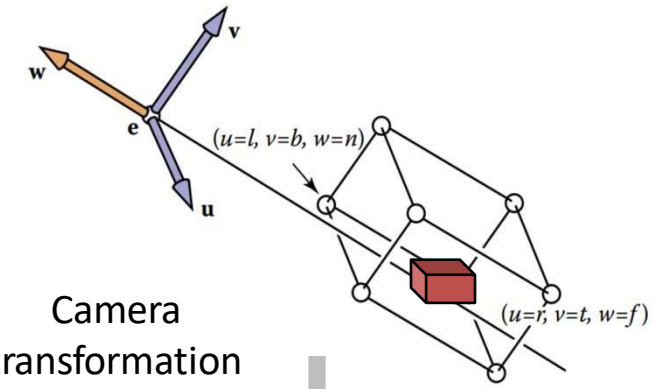
Canonical view volume

M_{vp}



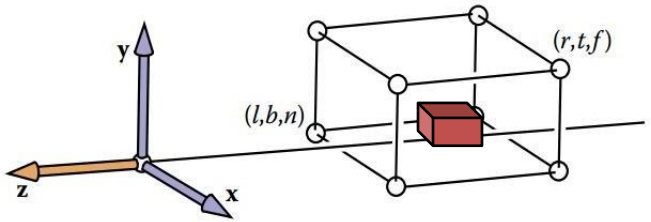
Screen

Summary (4/5)



Camera transformation

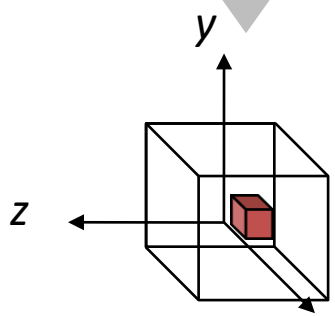
M_{cam}



Orthographic view volume

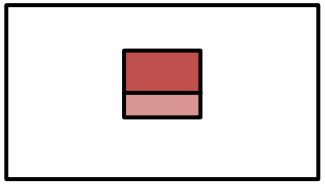
M_{orth}

$M = ?$



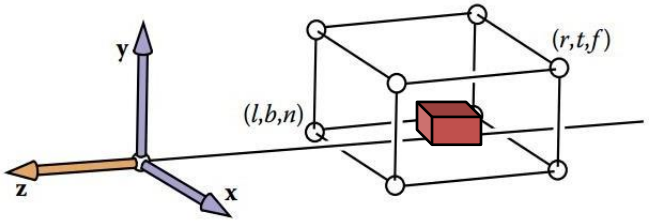
Canonical view volume

M_{vp}



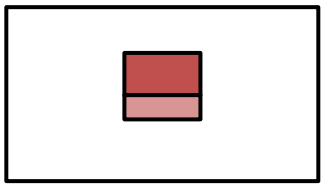
Screen

Summary (5/5)



Orthographic view volume

$M = M_{vp} * M_{orth} * M_{cam}$



Screen

Code: *Orth. to Screen v.2* (1/1)

Drawing many 3D lines with endpoints a_i and b_i:

```
Construct M_vp
Construct M_orth
Construct M_cam

M = M_vp*M_orth*M_cam

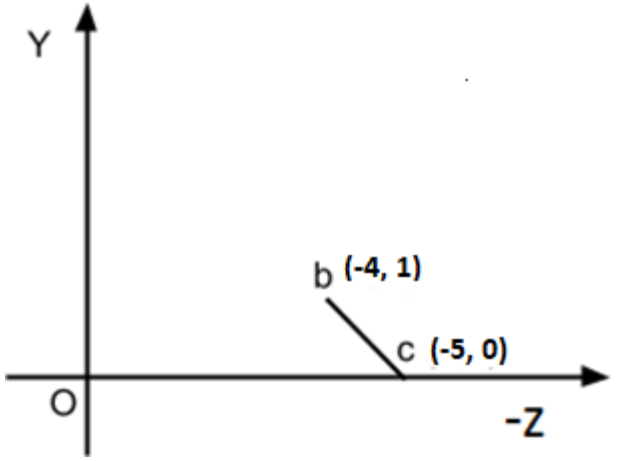
for each line segment (a_i, b_i) do:
      p = M*a_i
      q = M*b_i
      drawline (x_p, y_p, x_q, y_q)
```

Practice Problem - 1

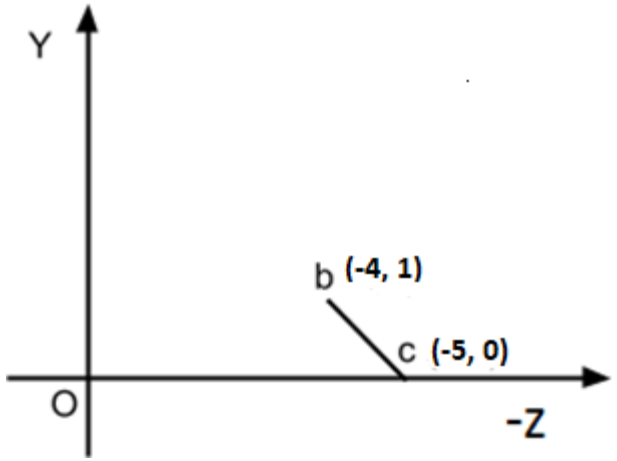
- Origin ***O*** and basis ***{z, y}*** construct a 2D canonical coordinate system where –z is the viewing direction. Within this, line ***bc*** is our model (P_{xy}). We want to view it from a new 2D camera (frame) with origin ***(-4, 8)*** looking downward.

(a) Determine *canonical-to-frame* matrix

(b) Calculate and plot P_{wv}.

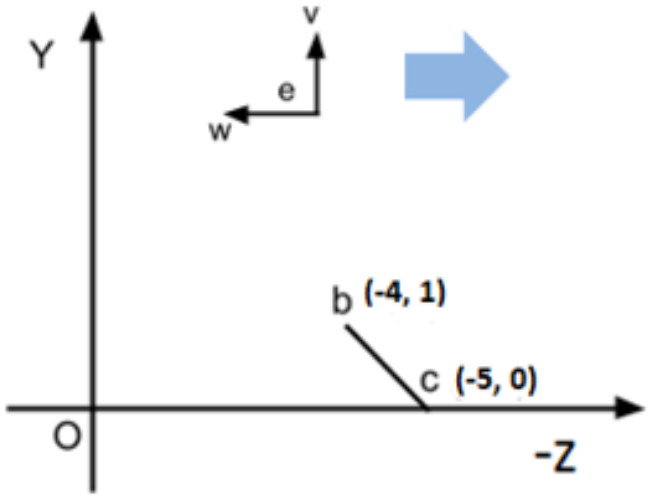


Practice Problem - 1



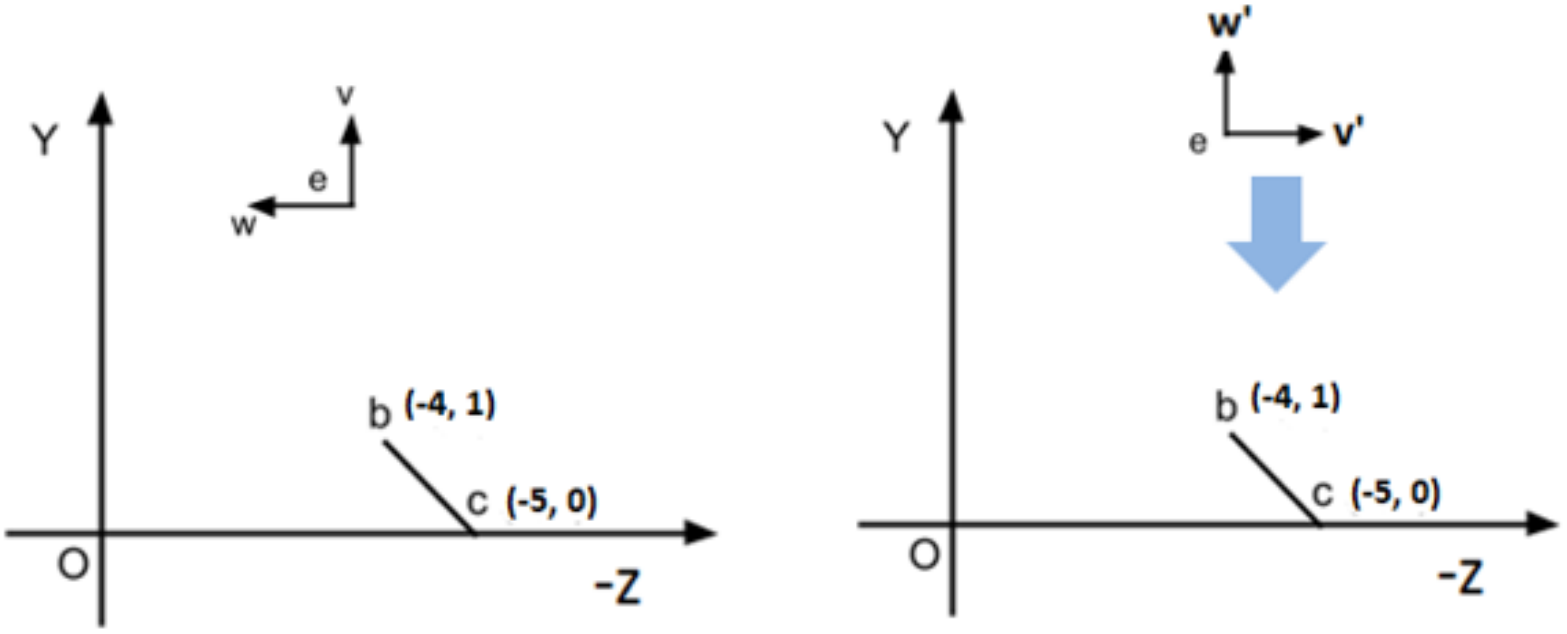
Practice Problem - 1

- e ≡ (-4,8); w = ? ; v = ?



Practice Problem - 1

- e ≡ (-4,8); w’ = ? ; v’ = ?

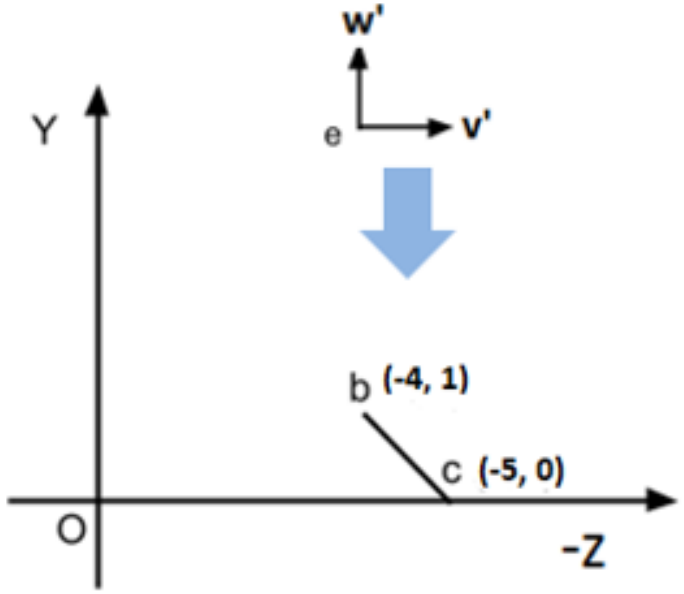


Practice Problem - 1

- e ≡ (-4,8); w’ = (0, 1) ; v’ = (-1, 0)

$$\mathbf{p}_{uv} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{e} \\ 0 & 0 & 1 \end{bmatrix}^{-1} \mathbf{p}_{xy}$$

$$\begin{bmatrix} u_p \\ v_p \\ 1 \end{bmatrix} = \begin{bmatrix} x_u & y_u & 0 \\ x_v & y_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_e \\ 0 & 1 & -y_e \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix}$$



Practice Problem - 2

a) Explain with appropriate example that the frame-to-canonical transformation can be expressed as a rotation followed by a translation.

b) Explain with appropriate example that canonical-to-frame transformation is a translation followed by a rotation; they are the inverses of the rotation and translation we used to build the frame-to-canonical matrix.

   – Hint: Fundamentals of Computer Graphics, Section 6.5